

Docker Compose Quickstart

Docker Docs - snapshot

Origem: <https://docs.docker.com/compose/gettingstarted/>

Copia de estudo exportada em 21/09/2026. Texto original em ingles; formatacao e paginacao locais. Imagens e navegacao omitidas; consulte a origem para diagramas. Apache License 2.0; credits em sources/.

Docker Compose Quickstart

This tutorial aims to introduce fundamental concepts of Docker Compose by guiding you through the development of a basic Python web application.

Using the Flask framework, the application features a hit counter in Redis, providing a practical example of how Docker Compose can be applied in web development scenarios. The concepts demonstrated here should be understandable even if you're not familiar with Python.

Prerequisites

Make sure you have:

Installed the latest version of Docker Compose

A basic understanding of Docker concepts and how Docker works

Step 1: Set up the project

Create a directory for the project:

```
$ mkdir compose-demo
$ cd compose-demo
```

Create app.py in your project directory and add the following:

```
import os
import redis
from flask import Flask

app = Flask(__name__)
cache = redis.Redis(
    host=os.getenv("REDIS_HOST", "redis"),
    port=int(os.getenv("REDIS_PORT", "6379")),
)

@app.route("/")
def hello():
    count = cache.incr("hits")
    return f"Hello from Docker! I have been seen {count} time(s).\n"
```

The app reads its Redis connection details from environment variables, with sensible defaults so it works out of the box.

Create requirements.txt in your project directory and add the following:

```
flask
redis
```

Create a Dockerfile :

```
# syntax=docker/dockerfile:1

# Build an image with the Python 3.12 image
FROM python:3.12-alpine
```

```
# Set the working directory to `/code`
WORKDIR /code

# Set environment variables used by the `flask` command
ENV FLASK_APP=app.py
ENV FLASK_RUN_HOST=0.0.0.0

# Install `gcc` and other dependencies
RUN apk add --no-cache gcc musl-dev linux-headers

# Copy `requirements.txt`
COPY requirements.txt .

# Install the Python dependencies
RUN pip install -r requirements.txt

# Copy the current directory `.` in the project to the workdir `.` in the image
COPY . .

EXPOSE 5000

# Set the default command for the container to `flask run --debug`
CMD ["flask", "run", "--debug"]
```

Make sure the file is named Dockerfile with no extension. Some editors add .txt automatically, which causes the build to fail.

For more information on how to write Dockerfiles, see the Dockerfile reference .

Create a .env file to hold configuration values:

```
APP_PORT=8000
REDIS_HOST=redis
REDIS_PORT=6379
```

Compose automatically reads .env and makes these values available for interpolation in your compose.yaml . For this example the gains are modest, but in practice, keeping configuration out of the Compose file makes it easier to:

Change values across environments without editing YAML

Avoid committing secrets to version control

Reuse values across multiple services

Create a .dockerignore file to keep unnecessary files out of your build context:

```
.env
*.pyc
__pycache__
redis-data
```

Docker sends everything in your project directory to the daemon when it builds an image. Without .dockerignore , that includes your .env file (which may contain secrets) and any cached Python bytecode. Excluding them keeps builds fast and avoids accidentally baking sensitive values into an image layer.

Step 2: Define and start your services

Compose simplifies the control of your entire application stack, making it easy to manage services, networks, and volumes in a single YAML configuration file.

Create compose.yaml in your project directory and paste the following:

```
services:
  web:
    build: .
    ports:
      - "${APP_PORT}:5000"
    environment:
      - REDIS_HOST=${REDIS_HOST}
```

```
- REDIS_PORT=${REDIS_PORT}

redis:
  image: redis:alpine
```

This Compose file defines two services:

The web service uses an image that's built from the Dockerfile in the current directory. It maps port 8000 on the host to port 5000 on the container where Flask listens by default.

The redis service uses a public Redis image pulled from the Docker Hub registry.

For more information on the compose.yaml file, see [How Compose works](#) .

Start up your application:

```
$ docker compose up
```

With a single command, you create and start all the services from your configuration file. Compose builds your web image, pulls the Redis image, and starts both containers.

Open <http://localhost:8000> . You should see:

```
Hello from Docker! I have been seen 1 time(s).
```

Refresh the page — the counter increments on each visit.

This minimal setup works, but it has two problems you'll fix in the next steps:

Startup race: web starts at the same time as redis . If Redis isn't ready yet, the Flask app fails to connect and crashes.

No persistence: If you run `docker compose down` followed by `docker compose up` , the counter resets to zero. `docker compose down` removes the containers, and with them any data written to the container's writable layer. `docker compose stop` preserves the containers so data survives, but you can't rely on that in production where containers are regularly replaced.

Stop the stack before moving on:

```
$ docker compose down
```

Step 3: Fix the startup race with health checks

To fix the startup race, Compose needs to wait until redis is confirmed healthy before starting web .

Update `compose.yaml` :

```
services:
  web:
    build: .
    ports:
      - "${APP_PORT}:5000"
    environment:
      - REDIS_HOST=${REDIS_HOST}
      - REDIS_PORT=${REDIS_PORT}
    depends_on:
      redis:
        condition: service_healthy

  redis:
    image: redis:alpine
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5
      start_period: 10s
```

The healthcheck block tells Compose how to test whether Redis is ready:

test is the command Compose runs inside the container to check its health. redis-cli ping connects to Redis and expects a PONG response — if it gets one, the container is healthy.

start_period gives Redis 10 seconds to initialize before health checks begin. Any failures during this window don't count toward the retry limit.

interval runs the check every 5 seconds after the start period has elapsed.

timeout gives each check 3 seconds to respond before treating it as a failure.

retries sets how many consecutive failures are allowed before Compose marks the container as unhealthy. With interval: 5s and retries: 5, Compose will wait up to 25 seconds before giving up.

Start the stack to confirm the ordering is fixed:

```
$ docker compose up
```

You should see something similar to:

```
[+] Running 2/2
✓ Container compose-demo-redis-1 Healthy 0.0s
```

Open <http://localhost:8000> to confirm the app is still working, then stop the stack before moving on:

```
$ docker compose down
```

Step 4: Enable Compose Watch for live updates

Without Compose Watch, every code change requires you to stop the stack, rebuild the image, and restart the containers. Compose Watch eliminates that cycle by automatically syncing changes into your running container as you save files.

Update compose.yaml to add the develop.watch block to the web service:

```
services:
  web:
    build: .
    ports:
      - "${APP_PORT}:5000"
    environment:
      - REDIS_HOST=${REDIS_HOST}
      - REDIS_PORT=${REDIS_PORT}
    depends_on:
      redis:
        condition: service_healthy
    develop:
      watch:
        - action: sync+restart
          path: .
          target: /code
        - action: rebuild
          path: requirements.txt

  redis:
    image: redis:alpine
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5
      start_period: 10s
```

The watch block defines two rules:

The sync+restart action watches your project directory (.) on the host. When a file changes, Compose copies any changed files into /code inside the running container, then restarts the container. Because the container restarts with the updated files already in place, Flask starts up reading the new code directly — no manual rebuild or restart needed.

The rebuild action on requirements.txt triggers a full image rebuild whenever you add a new dependency, since installing packages requires rebuilding the image, not just syncing files.

Start the stack with Watch enabled:

```
$ docker compose up --watch
```

Make a live change. Open app.py and update the greeting:

```
return f"Hello from Compose Watch! I have been seen {count} time(s).\n"
```

Save the file. Compose Watch detects the change and syncs it immediately:

```
Syncing service "web" after changes were detected
```

Refresh <http://localhost:8000> . The updated greeting appears without any restart and the counter should still be incrementing.

Stop the stack before moving on:

```
$ docker compose down
```

For more information on how Compose Watch works, see [Use Compose Watch](#) .

Step 5: Persist data with named volumes

Each time you stop and restart the stack the visit counter resets to zero. Redis data lives inside the container, so it disappears when the container is removed. A named volume fixes this by storing the data on the host, outside the container lifecycle.

Update compose.yaml :

```
services:
  web:
    build: .
    ports:
      - "${APP_PORT}:5000"
    environment:
      - REDIS_HOST=${REDIS_HOST}
      - REDIS_PORT=${REDIS_PORT}
    depends_on:
      redis:
        condition: service_healthy
    develop:
      watch:
        - action: sync+restart
          path: .
          target: /code
        - action: rebuild
          path: requirements.txt

  redis:
    image: redis:alpine
    volumes:
      - redis-data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5
      start_period: 10s

volumes:
  redis-data:
```

The redis-data:/data entry under redis.volumes mounts the named volume at /data , the path where Redis writes its data files. The top-level volumes key registers it with Docker so it persists between compose down and compose up cycles.

Start the stack with `docker compose up --watch` and refresh <http://localhost:8000> a few times to build up a count.

Tear down the stack with `docker compose down` and then bring it back up again with `docker compose up --watch`.

Open `http://localhost:8000` — the counter continues from where it left off.

Now reset the counter with `docker compose down -v`.

The `-v` flag removes named volumes along with the containers. Use this intentionally — it permanently deletes the stored data.

Step 6: Structure your project with multiple Compose files

As applications grow, a single `compose.yaml` becomes harder to maintain. The `include` top-level element lets you split services across multiple files while keeping them part of the same application.

This is especially useful when different teams own different parts of the stack, or when you want to reuse infrastructure definitions across projects.

Create a new file in your project directory called `infra.yaml` and move the Redis service and volume into it:

```
services:
  redis:
    image: redis:alpine
    volumes:
      - redis-data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5
      start_period: 10s

volumes:
  redis-data:
```

Update `compose.yaml` to include `infra.yaml`:

```
include:
  - path: ./infra.yaml
services:
  web:
    build: .
    ports:
      - "${APP_PORT}:5000"
    environment:
      - REDIS_HOST=${REDIS_HOST}
      - REDIS_PORT=${REDIS_PORT}
    depends_on:
      redis:
        condition: service_healthy
  develop:
    watch:
      - action: sync+restart
        path: .
        target: /code
      - action: rebuild
        path: requirements.txt
```

Run the application to confirm everything still works:

```
$ docker compose up --watch
```

Compose merges both files at startup. The web service can still reference redis by name because all included services share the same default network.

This is a simplified example, but it demonstrates the basic principle of `include` and how it can make it easier to modularize complex applications into sub-Compose files. For more information on `include` and working with multiple Compose files, see [Working with multiple Compose files](#).

Stop the stack before moving on:

```
$ docker compose down
```

Step 7: Inspect and debug your running stack

With a fully configured stack, you can observe what's happening inside your containers without stopping anything. This step covers the core commands for inspecting the resolved configuration, streaming logs, and running commands inside a running container.

Before starting the stack, verify that Compose has resolved your `.env` variables and merged all files correctly:

```
$ docker compose config
```

`docker compose config` doesn't require the stack to be running — it works purely from your files. A few things worth noting in the output:

`${APP_PORT}`, `${REDIS_HOST}`, and `${REDIS_PORT}` have all been replaced with the values from your `.env` file.

Short-form port notation (`"8000:5000"`) is expanded into its canonical fields (`target` , `published` , `protocol`).

The default network and volume names are made explicit, prefixed with the project name `compose-demo` .

The output is the fully resolved configuration, with any files brought in via `include` merged into a single view.

Use `docker compose config` any time you want to confirm what Compose will actually apply, especially when debugging variable substitution or working with multiple Compose files.

Now start the stack in detached mode so the terminal stays free for the commands that follow:

```
$ docker compose up -d
```

Stream logs from all services

```
$ docker compose logs -f
```

The `-f` flag follows the log stream in real time, interleaving output from both containers with color-coded service name prefixes. Refresh `http://localhost:8000` a few times and watch the Flask request logs appear. To follow logs for a single service, pass its name:

```
$ docker compose logs -f web
```

Press `Ctrl+C` to stop following logs. The containers keep running.

Run commands inside a running container

`docker compose exec` runs a command inside an already-running container without starting a new one. This is the primary tool for live debugging.

Verify environment variables are set correctly

```
$ docker compose exec web env | grep REDIS
```

```
REDIS_HOST=redis
REDIS_PORT=6379
```

Test that the web container can reach Redis using the service name as the hostname

```
$ docker compose exec web python -c "import redis; r = redis.Redis(host='redis'); print(r.ping())"
```

```
True
```

This uses the same `redis` library your app uses, so a `True` response confirms that service discovery, networking, and the Redis connection are all working end to end.

Inspect the live value of the hit counter in Redis

```
$ docker compose exec redis redis-cli GET hits
```

Where to go next

Explore the full list of Compose commands

Explore the Compose file reference

Check out the Learning Docker Compose video on LinkedIn Learning

Learn how to set environment variables in Compose

Learn how to package and distribute your Compose app